

High-order space–time parallel computing of the Navier–Stokes equations

Meiyuan Zhen, Xuan Liu, Xuejun Ding, Jinsheng Cai *

School of Aeronautics, Northwestern Polytechnical University, Xi'an, 710072, Shaanxi, China

ARTICLE INFO

Keywords:

Space–time parallel
Parallel-in-time
Integral deferred correction
High-order computing
Navier–Stokes equations

ABSTRACT

Space–time parallel methods can leverage modern parallel computing architectures to further accelerate numerical simulations where parallelizing only in space limits concurrency. In this paper, we develop a high-order space–time parallel computing method for solving the Navier–Stokes equations. The method is based on revisionist integral deferred correction (RIDC) algorithm proposed by Christlieb, Macdonald and Ong [SIAM J. Sci. Comput., 32 (2010), pp. 818–835]. This parallel-in-time algorithm yields high-order accurate results within a wall-clock time comparable to that of a forward or backward Euler simulation. We extend the RIDC algorithm to a space–time parallel method and apply the method to a compressible fluid solver using the message passing interface (MPI) framework. The global communicator is split into space and time communicators to manage parallel computing in spatial and temporal domains, respectively. Accuracy tests show that the method is high-order accurate. Numerical results encompassing both smooth flows and flows with discontinuities demonstrate that the proposed method can accelerate the computation robustly. Additionally, a three-dimensional space–time parallel computation of flow around a sphere is performed. The results indicate that RIDC has the potential to be used in the field of computational fluid dynamics.

1. Introduction

Spatial parallelism has proven to be highly effective in harnessing the high performance computing architecture to accelerate numerical simulations of fluid problems. However, parallel efficiency often decreases as more computational cores are used, leading to a bottleneck. As an alternative, researchers can consider parallel-in-time direction to take advantage of the surge in computational resources. The research of time-parallel methods is an active topic in the field of numerical methods. Although parallel-in-time (PinT) seems counter-intuitive, the earliest reported research dates back to 50 years ago [1], and many other PinT methods have proven to be potential. For comprehensive insights into this area, review articles [2,3] provide excellent introductions.

Following the categories in [4], there are three general approaches for time-parallel computation. One approach is parallel-across-the-problem, which decomposes a problem into several sub problems that can be computed in parallel [5,6]. The second approach is parallel-across-the-step, in which solutions at multiple time-grid points are computed simultaneously. Examples of this class of methods include the well-known parareal [7], parallel full approximation scheme in space and time (PFASST) [8], and multigrid reduction in time (MGRIT) [9]. These methods have been shown to successively accelerate numerical simulations in many studies [10–16]. However, one issue of such methods is that time-parallel computing of advection-dominated problems often shows slow convergence. Some strategies, such as adaptive mesh coarsening [17] and coarse-grid operator optimization [18], can improve the performance of MGRIT applied to simple model equations. However, in more complex systems, maintaining the

* Corresponding author.

E-mail address: caijsh@nwpu.edu.cn (J. Cai).

parallel efficiency of time-parallel methods remains a challenge. By contrast, the third approach, parallel-across-the-method, does not degenerate in advection-dominated problems. Another benefit of such methods is that they can achieve speedup even using a few number of processors.

Revisionist integral deferred correction (RIDC) [19], based on integral deferred correction (IDC) [20,21], is one of the parallel-across-the-method approaches. The IDC method is very similar to the spectral deferred correction (SDC) method [20], with the main difference being the ability of the latter to use any single-step integrator to solve the error equation. If one uses forward or backward Euler integrator to solve the error equation in IDC, then the method is equivalent to the SDC method. The most attractive feature of such deferred correction methods is that constructing high-order accurate methods is very simple, requiring only one correction to increase a certain degree of accuracy. As regards the efficiency, Xia et al. perform numerical experiments in [22] to compare semi-implicit SDC (which is equivalent to IDC in their work), additive Runge–Kutta and the exponential time differencing method. They conclude that the three methods are all competitive when solving ODEs from the local discontinuous Galerkin discretization of PDEs containing higher order spatial derivatives, with the SDC method having the advantage of easy implementation for arbitrary order of accuracy. Christlieb et al. show in [21] that high-order explicit IDC methods (such as the eighth-order IDC methods in the reference) match the efficiency of optimized Runge–Kutta methods. RIDC can perform IDC prediction and correction steps in parallel by increasing the number of intervals in one group, thereby achieving time-parallel acceleration. Subsequent work extend the method to implicit operators in [23], to adaptive step-size control in [24], and to the Navier–Stokes (NS) equations in [25]. Usually, they only perform parallel computing in time direction for small-scale problems. In [26], RIDC is combined with domain decomposition (DD) method to obtain a space–time parallel solver, which is applied to the calculation of the heat equation and the linear convection–diffusion equation, demonstrating the potential of applying RIDC to large-scale problems.

In this paper, RIDC algorithm is extended to a space–time parallel method to further accelerate high-order computing of the NS equations when parallel-only-in-space methods are unable to scale further. Research of high-order methods is one of the most active research topics in the field of computational fluid dynamics (CFD), as they are more efficient than low-order methods when high accuracy is required. RIDC is designed to be a high-order method that achieves parallel acceleration, and thus is very suitable for application in CFD. Another advantage of RIDC is that it can deal with convection-dominated problems without any difference with parabolic problems. Therefore, we extend RIDC algorithm to perform space–time parallel computing of the NS equations. Using the message passing interface (MPI) parallel computing framework, we employ communicators to manage the parallel-in-time and -space directions separately. Parallelism in space is achieved by using ghost cells to exchange messages, which is a widely adopted technique in the CFD community. For the numerical results, we first focus on the time-parallel performance of the proposed method, and perform time-parallel computing of model equations and several inviscid fluid problems to verify the temporal accuracy and to study the speedup achieved by the method when no spatial parallelism is used. We then conduct space–time parallel computing of two viscous problems to study the performance of the method when both spatial and temporal parallelism are utilized. A scaling test is provided to demonstrate that space–time parallel RIDC can indeed break through the space-parallel bottleneck and further accelerate the computation.

The remainder of this paper is organized as follows. Section 2 introduces the IDC method and the space–time parallel RIDC. Section 3 provides time-parallel computing results of model equations. In Section 4, we perform time-parallel and space–time parallel computing of the NS equations. Finally, conclusions are given in Section 5.

2. IDC and space–time parallel RIDC methods

Since RIDC is based on the IDC method, we provide an overview of IDC first. Then we explain RIDC algorithm and extend it to the space–time parallel RIDC algorithm.

2.1. The IDC method

We consider an initial-value problem (IVP) of the form

$$y'(t) = f(t, y), \quad y(0) = y_0, \quad t \in [0, T]. \quad (1)$$

The time domain, $[0, T]$, is divided into N intervals,

$$0 = t_0 < t_1 < \cdots < t_n < \cdots < t_N = T, \quad (2)$$

and each interval, $[t_n, t_{n+1}]$, is further discretized into M sub-intervals including $M + 1$ quadrature nodes,

$$t_n = t_{n,0} < t_{n,1} < \cdots < t_{n,m} < \cdots < t_{n,M} = t_{n+1}. \quad (3)$$

Fig. 1 shows the time grid and quadrature nodes in an interval.

Denote the approximation of the exact solution $y(t)$ as $u(t)$. The error of the approximation is

$$e(t) = y(t) - u(t). \quad (4)$$

The residual (sometimes termed as the defect) of Eq. (1) is defined as

$$r(t) = u'(t) - f(t, u). \quad (5)$$

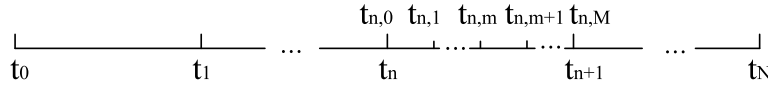


Fig. 1. The IDC method uses a time grid containing N intervals, and each interval is divided into M sub-intervals.

IDC advances the solution interval by interval, and in each of these intervals, the solution is advanced sub-interval by sub-interval. We describe the procedure of the IDC method when marching the solution in the interval $[t_n, t_{n+1}]$. The size of this interval is Δt_n and the size of the sub-interval $[t_{n,m}, t_{n,m+1}]$ is $\Delta t_{n,m}$. The approximation at the node $t_{n,m}$ is denoted as $u_{n,m}$. Since we only consider the interval $[t_n, t_{n+1}]$, without causing confusion, we drop the subscript n , so that $t_{n,m} =: t_m$, the error at the node $t_{n,m}$ being $e_{n,m} =: e_m$ and the approximation $u_{n,m} =: u_m$.

The IDC method consists of two stages.

- (prediction level) Use a k th order time integrator to obtain the initial approximation $u^{[0]}$.
- (correction loop) Correct the approximation to improve the accuracy iteratively. For $p = 0, \dots, P-1$ (P is the number of correction levels)

1. Compute the residual $r^{[p]}$ using Eq. (5). Differentiate the error Eq. (4) and incorporate the residual,

$$e'(t) = y'(t) - u'(t) = f(t, u + e) - f(t, u) - r(t), \quad e(0) = 0. \quad (6)$$

This equation is also an IVP, which can be formulated in an integral form [20] for better stability,

$$\left(e + \int_0^t r(\tau) d\tau \right)' = f(t, u + e) - f(t, u). \quad (7)$$

2. Solve the IVP (7) using a time integrator, for example, the forward Euler method. The error after the p th correction at the node t_{m+1} , denoted by $e_{m+1}^{[p]}$, is obtained by

$$e_{m+1}^{[p]} = e_m^{[p]} + \Delta t_m \left(f(t_m, u_m^{[p]} + e_m^{[p]}) - f(t_m, u_m^{[p]}) \right) - \int_{t_m}^{t_{m+1}} r^{[p]}(\tau) d\tau. \quad (8)$$

3. Update the numerical approximation $u^{[k+1]} = u^{[k]} + e^{[k]}$.

In practical implementation, the residual is not computed explicitly. We substitute the residual equation (5) into Eq. (8) to directly compute the approximation,

$$u_{m+1}^{[p+1]} - u_m^{[p+1]} = \Delta t_m \left(f(t_m, u_m^{[p+1]}) - f(t_m, u_m^{[p]}) \right) + \int_{t_m}^{t_{m+1}} f(\tau, u^{[p]}) d\tau. \quad (9)$$

Similarly, if the first-order backward Euler is used, the calculation is implicit,

$$u_{m+1}^{[p+1]} - u_m^{[p+1]} = \Delta t_m \left(f(t_{m+1}, u_{m+1}^{[p+1]}) - f(t_{m+1}, u_{m+1}^{[p]}) \right) + \int_{t_m}^{t_{m+1}} f(\tau, u^{[p]}) d\tau. \quad (10)$$

2.2. The space-time parallel RIDC algorithm

To illustrate the distinctions between IDC and RIDC, we delineate the time-stepping procedure in Fig. 2. The numbers within the circles in Fig. 2 each indicate a prediction or a correction step. It is seen in Fig. 2(a) that IDC performs the prediction and correction steps sequentially. In contrast, RIDC concurrently advances the steps after the initial start-up procedure. Fig. 2(b) illustrates that the prediction and all correction steps can be computed in parallel starting from step twelve. The preceding eleven steps constitute the start-up procedure.

The reason RIDC can achieve speedup is that RIDC algorithm relaxes the requirement of iterating P correction loops completely in each interval of P sub-intervals. Instead, it allows each interval to contain M sub-intervals, where $M \gg P$. It is seen in Fig. 2(a) that each interval of the fourth-order IDC contains only three sub-intervals, while the fourth-order RIDC in Fig. 2(b) employs a substantially larger number of sub-intervals in an interval. The utilization of a greater number of sub-intervals in an interval leads to improved parallel efficiency. However, an excessive number of sub-intervals may give rise to error accumulation. Research conducted in [19] reveals that M between 40 and 100 leads to the minimum error for the cases in the article, while the error is problem-dependent.

A technique can be used to mitigate the memory requirement and increase parallel efficiency. In order to advance the approximation (using Eq. (9) or Eq. (10)), the original IDC uses a full quadrature stencil and an additional node for either prediction or correction levels, seen Fig. 3(a). Christlieb et al. [19] discover that the quadrature stencil can be different over the levels. Specifically, to compute the integral term in Eq. (9) or Eq. (10) during the first correction level, the fourth-order RIDC only requires a quadrature stencil with two nodes. Similarly, three and four quadrature nodes are required during the second and the third correction

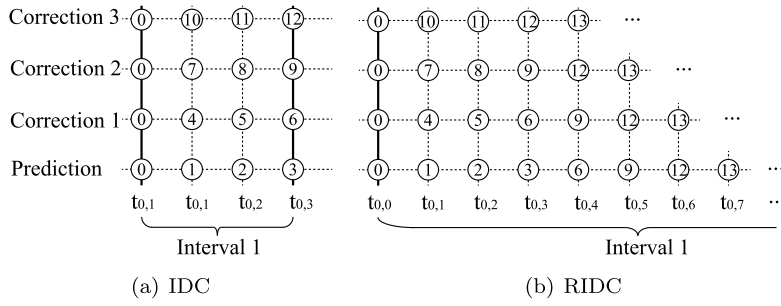


Fig. 2. Schematic diagram of the execution sequence of prediction and correction steps for IDC and RIDC. The numbers within circles each indicate a prediction or a correction step.

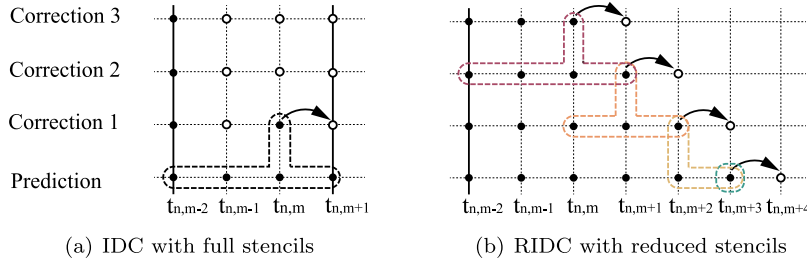


Fig. 3. The stencils used for IDC and RIDC prediction and correction levels, which are indicated by black dots surrounded by dashed lines. The fourth-order IDC employs the same full stencil while RIDC utilizes four different stencils for prediction and three corrections.

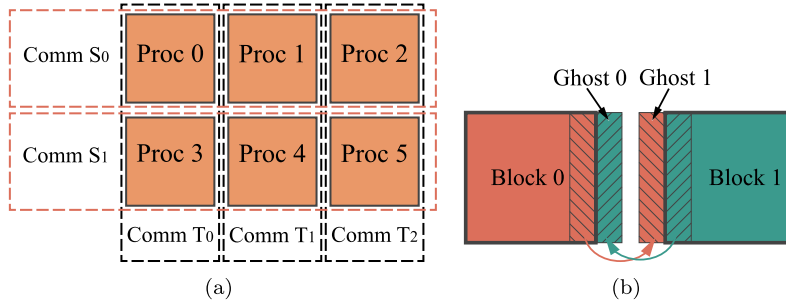


Fig. 4. Space and time communicators manage the communication during space-time parallel computing. (a) Six processors are split into two groups in space direction and three groups in time direction. As a result, two space communicators and three time communicators are generated. (b) Communications between two spatial zones using ghost cells.

level, respectively, which is depicted in Fig. 3(b). Even though smaller stencils are utilized, RIDC using the reduced stencils still achieves the same order of accuracy as the IDC using full stencils.

RIDC achieves speedup through parallelism in time. In order to leverage more computer resources to accelerate the simulation of larger-scale problems, we extend the RIDC algorithm to a space-time parallel method.

We implement the space-time parallel algorithm using the MPI framework. Firstly, based on the required order of accuracy and the number of processes at hand, the global MPI communicator is divided into several space and time communicators. The space communicators are responsible for sending and receiving messages required for spatial parallelism, while the time communicators are in charge of transmitting messages for time-parallel computing. We use Fig. 4 to describe the space-time parallel RIDC algorithm, which will be referred to as RIDC-ST latter for convenience. In order to perform space-parallel computing, the global spatial grid is split into multiple blocks, for instance, the two blocks in Fig. 4(b). Since the spatial solver (using finite difference method) in this paper evolves the data outside block boundaries, the data on those cells (which are termed as ghost cells, see Fig. 4(b)) has to be exchanged near a boundary before the computing of f . Specifically in Fig. 4(a), the space communicators S_0 and S_1 have to exchange the messages near the boundary between processor 0 and 3, also between processor 1 and 4, and between 2 and 5. In terms of time parallelism, time communicator T_0 manages the data of the prediction level, T_1 manages that of the first correction level, and so forth. Since solving Eq. (9) and Eq. (10) requires the f vectors of the previous level (which is the prediction level or the previous correction level), the current level has to receive the message from the previous level. Specifically in Fig. 4(a),

time communicator T_0 sends f vectors and T_1 receives them. Similarly, time communicator T_2 receives the data from T_1 . When all messages are ready, the six processors in Fig. 4(a) advance concurrently. To reduce the communication overhead in the practical implementation, non-blocking communications are used.

The space–time parallel RIDC algorithm using the Euler time-advancing method is summarized in Algorithm 1 and Algorithm 2. The number P in two algorithms is the number of correction levels. MPI communication operations are ignored in Algorithm 1 for clarity, but are written in Algorithm 2 to illustrate the difference between RIDC-ST algorithm and the original time-parallel RIDC algorithm (Algorithm 2 in [27]).

Algorithm 1 RIDC-ST Start-up Procedure

```

1: for  $p = 1$  to  $P$  do
2:   Advance previous levels  $0, \dots, (p-1)$  for one step in parallel.
3:   Advance current level for  $(p-1)$  steps.
4:   Advance levels  $0, \dots, p$  for one step in parallel.
5: end for

```

Algorithm 2 RIDC-ST Parallel Stage pseudocode

```

1: for  $p = 0$  to  $P$ , parallelly do
2:   if  $p = 0$ , and current time  $t < t_M$  then
3:     1. Time communicator Comm  $T_0$  sends  $f^{[0]}$  vectors at the quadrature nodes to Comm  $T_1$ .
4:     2. Advance the solution for one step using the forward or backward Euler method. If the backward Euler method is utilized, ghost messages are exchanged repeatedly by space communicators.
5:     3. Use space communicators to exchange ghost messages, and then compute  $f_{m+1}^{[0]}$ .
6:   else if  $p > 0$ , and current time  $t < t_M$  then
7:     if  $p < P$  then
8:       Time communicator Comm  $T_p$  sends  $f^{[p]}$  vectors at the quadrature nodes to Comm  $T_{p+1}$ .
9:     end if
10:    1. Time communicator Comm  $T_p$  receives  $f^{[p-1]}$  vectors from Comm  $T_{p-1}$ .
11:    2. Advance the Equation (9) or Equation (10) for one step. If Equation (10) is utilized, ghost messages are exchanged repeatedly by space communicators.
12:    if  $p < P$  then
13:      Use space communicators to exchange ghost messages, and then compute  $f_{m+1}^{[p]}$ .
14:    end if
15:  end if
16: end for

```

In this paper, we only consider the implicit method in Eq. (10). The p th order RIDC-ST using M sub-intervals are denoted by RIDC_M^p -ST. If the subscript M is not shown, it simply means a p th order RIDC-ST method.

3. Results of model equations

In this section, we focus on the temporal accuracy and parallel-in-time performance of RIDC-ST for simple cases. Model equations are computed in parallel, but only in time direction. Fully space–time parallel computation will be performed in several cases in Section 4. For all cases in this section, the 5th-order WENO-JS scheme in [28] is utilized for spatial discretization. This nonlinear scheme is designed to handle advective terms and even shocks, rendering it well-suited for the problems discussed herein.

3.1. The order of accuracy

The linear advection equation with periodic boundaries is given as

$$\begin{aligned} \frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} &= 0, \quad (x, t) \in [-1, 1] \times [0, 2] \\ u(x, 0) &= \sin(\pi x - \frac{\sin(\pi x)}{\pi}), \end{aligned} \tag{11}$$

which has the analytic solution $u(x, t) = u(x - t, 0)$.

To verify the temporal accuracy of the RIDC-ST, we compute the linear advection equation using various time grids and different numbers of correction levels. For each number of correction levels, we employ a range of sub-intervals M , varying from 8 to 128. The end time is fixed at $t_e = 0.1$, and the spatial grid consists of 2000 points. The computing results are collected in Table 1 and Table 2. Values in “Order” columns in the tables give the calculated order of accuracy according to the simulation error. It is seen that all results show the expected order of accuracy except that the RIDC_M^5 -ST case using $M = 128$ exhibits a calculated order of

Table 1

L_2 -norm error and convergence orders of the computation of the linear advection equation using the 2nd-order RIDC-ST and the 3rd-order RIDC-ST with various numbers of sub-intervals.

M	RIDC ² -ST		RIDC ³ -ST	
	Error	Order	Error	Order
8	1.40994e-05	–	6.67909e-07	–
16	3.56173e-06	1.985	8.87274e-08	2.912
32	8.94550e-07	1.993	1.14208e-08	2.958
64	2.24120e-07	1.997	1.44825e-09	2.979
128	5.60882e-08	1.999	1.82318e-10	2.990

Table 2

L_2 -norm error and convergence orders of the computation of the linear advection equation using the 4th-order RIDC-ST and the 5th-order RIDC-ST with various numbers of time points.

M	RIDC ⁴ -ST		RIDC ⁵ -ST	
	Error	Order	Error	L_2 Order
8	2.83481e-08	–	1.43708e-09	–
16	1.79981e-09	3.977	4.96297e-11	4.856
32	1.12769e-10	3.996	1.64599e-12	4.914
64	7.04250e-12	4.001	5.61228e-14	4.874
128	4.34719e-13	4.018	8.77282e-15	2.677

2.667, which is much smaller than the expected five. This is because when 128 time points are used for the 5th-order RIDC-ST, the error becomes so minimal that the spatial error becomes dominant. To mitigate the influence of the spatial error, we conducted a test of 5th-order RIDC-ST using 4000 spatial grid while keep other parameters unchanged. The results show a 5th order of accuracy even when $M = 128$ (results not shown herein).

3.2. Two-dimensional Buckley–Leverett equation

Before examining the performance of RIDC-ST, we solve the 2D Buckley–Leverett equation to show that IDC (and thus RIDC) methods can deal with the jump in the field variables. The 2D Buckley–Leverett equation is given as

$$\frac{\partial u}{\partial t} + \frac{\partial f}{\partial x} + \frac{\partial g}{\partial y} = \epsilon \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \quad (12)$$

where $\epsilon = 0.01$, and the flux functions are given as

$$f(u) = \frac{u^2}{u^2 + (1 - u)^2}, \quad g(u) = (1 - 5(1 - u)^2)f(u). \quad (13)$$

The problem is solved in a domain of $[-1.5, 1.5] \times [-1.5, 1.5]$, which is discretized by 200×200 spatial grid points. The end time $t_e = 0.5$. The initial condition is

$$u(x, y, 0) = \begin{cases} 1, & x^2 + y^2 < 0.5, \\ 0, & \text{otherwise.} \end{cases} \quad (14)$$

We use the third-order IDC (IDC3) method to solve the problem, and compared the results with what obtained by the optimal third-order strong-stability-preserving Runge–Kutta (SSP-RK3) method in [29]. The results are shown in Fig. 5. It is seen in Fig. 5(a) that the result obtained by IDC3 is accurate and almost the same as that of SSP-RK3, both not showing any oscillation or overshoot.

3.3. Linear advection equation

The RIDC-ST methods have proven to achieve the expected orders of accuracy in Section 3.1, and are shown to be considerably stable in Section 3.2. Here, we examine the speedup and efficiency achieved by RIDC-ST for the linear advection problem described in Eq. (11). The speedup is over the IDC methods of the same order, which solve the problem step by step. The wall time consumed by the IDC methods is show in “Seq. time” column in tables. We set the number of intervals and the number of sub-intervals in each interval as $N = 64$ and $M = 64$, respectively. The number of spatial grid points is set to $N_x = 1000$. As shown in Table 3, when $p > 1$, all RIDC-ST methods achieve speedup over the corresponding sequential one. However, the efficiency decreases when the order of accuracy increases. When five processors are used for the fifth-order RIDC-ST, it achieves a speedup of approximately four, with an efficiency of 78.4%. From another perspective, we can compare the wall time consumed by the RIDC¹-ST and RIDC⁵-ST. It demonstrates that by using only around 1.27 times the wall time required by RIDC¹-ST, RIDC⁵-ST achieves fifth order of accuracy, rather than only first order of accuracy. This observation is motivating and underscores the gains realized through the application of RIDC-ST.

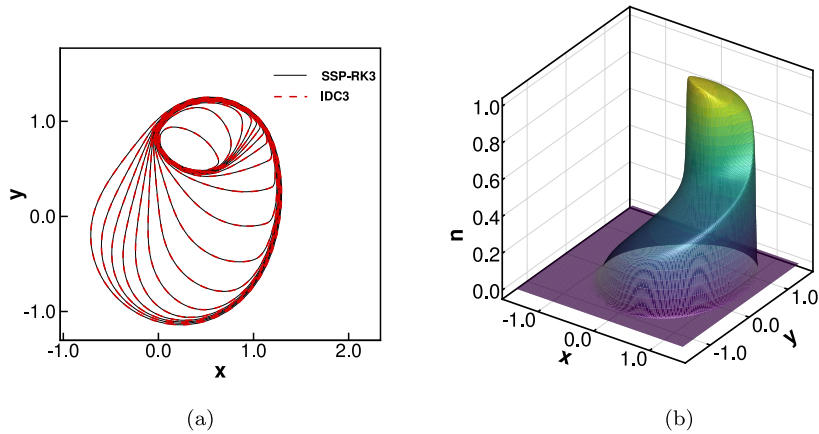


Fig. 5. Results obtained by the third-order IDC method and SSP-RK3 for solving the 2D Buckley–Leverett equation. (a) Contour lines of u . (b) 3D display of u .

Table 3

The linear advection problem. Wall time consumed by RIDC-ST of different orders from first order to fifth order (denoted by p). “Seq. time” column in the table indicates the time consumed by the IDC methods of p th order.

p	Time (s)	Seq. time (s)	Speedup	Efficiency (%)
1	499.24	–	–	–
2	504.89	991.70	1.96	98.2
3	527.88	1492.76	2.83	94.3
4	566.81	1991.77	3.51	87.8
5	635.90	2491.67	3.92	78.4

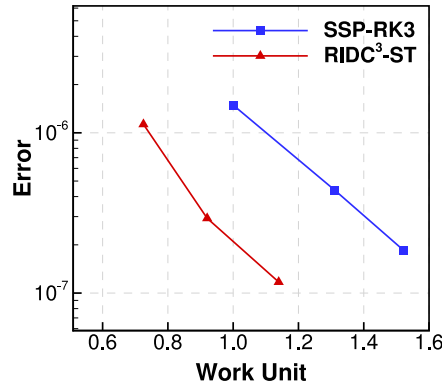


Fig. 6. Linear advection problem. The accuracy achieved by the third-order RIDC-ST and SSP-RK3 by using certain amounts of work unit.

Although high-order RIDC-ST achieves speedup over the corresponding IDC method, a comparison between RIDC-ST and the more widely-used SSP-RK would better illustrate the practical value of the former. We solve the linear advection problem (11) by the third-order RIDC-ST and the optimal implicit SSP-RK3 in [29] using 1000 spatial grid points. SSP-RK3 uses 200 time steps, and RIDC³-ST adopts 15 time steps ($N = 15$) with $M = 40$. Since each of the SSP-RK3 step consists of three stages, the total number of stages of the 200 SSP-RK3 steps is 600, which is exactly the same as the number of the sub-intervals of RIDC³-ST using $N = 15$ and $M = 40$. Thus, the comparison is considered to be fair.

The time consumed by SSP-RK3 to solve the problem is 251 s, while RIDC³-ST uses 231 s, which is slightly faster than SSP-RK3 by 8%. In terms of the resulting error, the L_2 -norm of the error of SSP-RK3 is 1.49×10^{-6} , while that of RIDC³-ST is 2.93×10^{-7} . It means that RIDC³-ST obtains more accurate results than SSP-RK3 by using slightly less wall time. To be more convincing, we compare the accuracy achieved by the two methods for a given amount of work. The two methods using various numbers of time steps are utilized to solve the problem. We specify the wall time consumed by SSP-RK3 of 200 time steps as one work unit, and then normalize other data. The plot of accuracy by work unit is drawn in Fig. 6. It is observed that for a given amount of work, RIDC³-ST provides more accurate result than SSP-RK3 by almost an order of magnitude, at least for this problem. A similar comparison is conducted for the isentropic vortex problem in Section 4.2.

Table 4

The inviscid Burgers' problem. Wall time consumed by RIDC of different orders from first order to fifth order. "Seq. time" column in the table indicates the time consumed by the IDC methods of p th order.

p	Time (s)	Seq. time (s)	Speedup	Efficiency (%)
1	440.57	—	—	—
2	441.49	864.12	1.96	97.9
3	463.02	1297.42	2.80	93.4
4	503.38	1728.55	3.43	85.8
5	562.68	2164.39	3.85	76.9

3.4. Inviscid Burgers' equation

The inviscid Burgers' equation is given as

$$\begin{aligned} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} &= 0, \quad (x, t) \in [0, 2] \times [0, 2] \\ u(x, 0) &= \sin(\pi x). \end{aligned} \quad (15)$$

This problem is considered herein to examine the performance of RIDC-ST in nonlinear problems. We employ 1000 spatial grid points for discretization. Both N and M are set to be 64. The results for this problem are presented in Table 4. Speedup achieved by RIDC-ST in this problem closely resembles the one observed in the linear advection problem. Likewise, the parallel efficiency decreases as more processors are used for computing higher-order accurate solutions. When five processors are utilized, the parallel efficiency becomes 76.9%. Notably, high-order accurate results can be obtained with a wall-clock time that is comparable to that of a single backward Euler simulation. This highlights the accuracy gains that RIDC-ST offers in addressing nonlinear problems.

4. Results of Navier–Stokes equations

Navier–Stokes equations in vector form are written as

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{E}}{\partial x} + \frac{\partial \mathbf{F}}{\partial y} + \frac{\partial \mathbf{G}}{\partial z} = \frac{\partial \mathbf{E}_v}{\partial x} + \frac{\partial \mathbf{F}_v}{\partial y} + \frac{\partial \mathbf{G}_v}{\partial z}, \quad (16)$$

where $\mathbf{U}$ is the conservative variable vector, $\mathbf{E}$, $\mathbf{F}$ and $\mathbf{G}$ are inviscid fluxes in x , y and z directions, and $\mathbf{E}_v$, $\mathbf{F}_v$ and $\mathbf{G}_v$ are viscous fluxes in three directions, respectively. The conservative variable vector $\mathbf{U}$ and fluxes are specifically defined as

$$\begin{aligned} \mathbf{U} &= \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ \rho w \\ \rho E \end{pmatrix}, \mathbf{E} = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ \rho uw \\ \rho Eu + pu \end{pmatrix}, \mathbf{F} = \begin{pmatrix} \rho v \\ \rho vu \\ \rho v^2 + p \\ \rho vw \\ \rho Ev + pv \end{pmatrix}, \mathbf{G} = \begin{pmatrix} \rho w \\ \rho wu \\ \rho wv \\ \rho w^2 + p \\ \rho Ew + pw \end{pmatrix}, \\ \mathbf{E}_v &= \begin{pmatrix} 0 \\ \tau_{11} \\ \tau_{12} \\ \tau_{13} \\ \varphi_1 \end{pmatrix}, \mathbf{F}_v = \begin{pmatrix} 0 \\ \tau_{21} \\ \tau_{22} \\ \tau_{23} \\ \varphi_2 \end{pmatrix}, \mathbf{G}_v = \begin{pmatrix} 0 \\ \tau_{31} \\ \tau_{32} \\ \tau_{33} \\ \varphi_3 \end{pmatrix}, \end{aligned} \quad (17)$$

where ρ, u, v, w , and p represent density, flow velocity in three directions and pressure. ρE is the total energy including internal energy and kinetic energy. By using the Einstein summation convention, viscous stress tensor τ_{ij} and heat flux φ_i are expressed as

$$\begin{aligned} \tau_{ij} &= \mu \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} - \frac{2}{3} \delta_{ij} \frac{\partial u_k}{\partial x_k} \right), \\ \varphi_i &= u_j \tau_{ij} + k \frac{\partial T}{\partial x_i}, \end{aligned} \quad (18)$$

where μ is the dynamic viscosity coefficient and k is the heat conduction coefficient. The system of equations is closed by the ideal gas equation with $\gamma = 1.4$.

For the discretization of Eq. (16), fifth-order cell-centered WCNS (Weighted Compact Nonlinear Schemes) [30] scheme is used to discretize convective terms. This scheme involves an initial interpolation of variables to cell faces using a high-order interpolation method. Subsequently, cell-face fluxes are computed using flux difference splitting methods, and the derivatives of convective fluxes are determined through a linear finite difference approach. The scheme is able to capture shocks and to deal with complex geometries. For more details about WCNS, readers are referred to [30,31]. In this work, AUSMD/V (Advection Upstream Splitting Method) scheme [32] is utilized to compute the flux at the cell faces. Viscous terms are discretized by sixth-order central difference scheme. We use the dual-time stepping method [33] to solve unsteady problems. The method introduces a pseudo time to facilitate solution convergence during each physical time step. The dual-time stepping method can be simply understood as a relaxed Newton iteration method. Implicit method is used for time advancing, and the generated linear system is solved by DADI (Diagonal Alternative Direction Implicit) method [34].

Table 5

Error and convergence rates of the computation of the isentropic vortex problem using a set of spatial grids of different resolutions.

Resolution	L_2 error	L_2 order (s)	L_∞ error	L_∞ order
48×48	1.50921774e-10	–	2.01791950e-09	–
96×96	4.80369826e-12	4.97	6.75446366e-11	4.90
192×192	1.46288271e-13	5.04	1.92468264e-12	5.13
384×384	6.54063972e-15	4.48	8.68194405e-14	4.47

Table 6

L_2 -norm error and convergence orders of the computation of the isentropic vortex problem using the 1st-order RIDC-ST and the 2nd-order RIDC-ST with various numbers of sub-intervals M .

M	RIDC ¹ -ST		RIDC ² -ST	
	Error	Order	Error	Order
1	9.25011e-04	–	1.35936e-05	–
2	4.78155e-04	0.95	3.34338e-06	2.02
4	2.42045e-04	0.98	8.27159e-07	2.02
8	1.21642e-04	0.99	2.05640e-07	2.00

Table 7

L_2 -norm error and convergence orders of the computation of the isentropic vortex problem using the 3rd-order RIDC-ST and the 4th-order RIDC-ST with various numbers of sub intervals M .

M	RIDC ³ -ST		RIDC ⁴ -ST	
	Error	Order	Error	Order
1	7.45386e-07	–	4.60901e-08	–
2	9.96838e-08	2.90	3.10037e-09	3.89
4	1.27938e-08	2.96	1.98482e-10	3.97
8	1.61677e-09	2.98	2.53782e-11	2.96

4.1. Accuracy test

The numerical solution of the Navier–Stokes equations contains two types of errors, temporal discretization error and spatial discretization error. In order to test the accuracy of the proposed RIDC-ST, we perform accuracy tests in time and space directions independently.

We consider a problem from Yee et al. [35], where an isentropic vortex is convected with an inviscid free stream. The computations are performed on the domain of $[0, 0] \times [10, 10]$. Four boundaries are all set to periodic boundaries, so that the vortex advects along the diagonal of the computational domain and reaches the initial position after each period. The exact solution of the problem is simply the advection of the initial flow field, thus the error can be easily computed at any time. The initial field is given by

$$\rho = \left[1 - \frac{(\gamma - 1)\beta^2}{8\gamma\pi^2} e^{1-r^2} \right]^{\frac{1}{\gamma-1}}, \quad r^2 = \bar{x}^2 + \bar{y}^2,$$

$$(u, v) = (1, 1) + \frac{\beta}{2\pi} e^{\frac{1-r^2}{2}} (-\bar{y}, \bar{x}),$$

$$p = \rho^\gamma,$$

where β is the vortex strength and is set to 5. $\bar{x} = x - x_c$, and $\bar{y} = y - y_c$, where (x_c, y_c) is the center of the vortex at the initial time. The entire flow field is isentropic and $p/\rho^\gamma = 1$.

We first validate the accuracy of the spatial solver. In order to avoid the influence of the error due to time discretization, we use a very small time-stepping size, $dt = 1e-6$, and advance the solution for only one step. Third-order TVD Runge–Kutta method [36] is used for time stepping. A set of spatial grids of different resolutions are utilized to compute errors and the order of accuracy. The resulting error and convergence rates are shown in Table 5. It is seen that both L_2 -norm and L_∞ -norm errors show approximation fifth order convergence rate, which indicates that the spatial discretization scheme, fifth-order WCNS scheme, achieves the expected order of accuracy.

Similarly, we verify the temporal accuracy of the RIDC-ST by using different time-stepping sizes but the same spatial grid. To avoid the influence of the error due to spatial discretization, a fine spatial grid of the size 1536×1536 is employed. The fine grid is partitioned into 16 blocks and distributed to 16 processors. Therefore, the RIDC-ST methods use 16 to 64 processors in total for space–time parallel computing, 16 in spatial direction and 1 to 4 in temporal direction. We set N to one, and M from one to eight. The L_2 norm of density is employed to calculate orders of accuracy. The resulting errors and convergence rates using RIDC-ST of different orders are shown in Table 6 and Table 7.

RIDC-ST of four different orders all achieve the expected accuracy. Only RIDC⁴-ST using $M=8$ shows a convergence rate of 2.96, which is less than the expected number of 4. This is because the error from time discretization is so small that the spatial

discretization error comes into play and pollutes the accuracy. This is proved by refining the spatial grid and checking the order of accuracy again.

4.2. Numerical results

In this section, we first test the stability property of the IDC methods by solving 1D shock tube problems where the shock strength is varied from moderate to large. Then, we study the parallel performance of the RIDC-ST in flow problems. Two inviscid cases and two viscous problems are computed in parallel, including a three-dimensional flow problem.

One-dimensional shock tube problems

Before we study the parallel performance of RIDC-ST, we test whether IDC methods can deal with discontinuities in flow problems. Specifically, we study three shock tube problems with different shock strength, which are Lax problem [37], Sod problem [38], and the right half of Woodward and Colella problem [39].

The initial condition for Lax problem is

$$(\rho, u, p) = \begin{cases} (0.445, 0.698, 3.528) & 0 \leq x < 0.5 \\ (0.5, 0, 0.5710) & 0.5 \leq x \leq 1, \end{cases} \quad (19)$$

and the final time is $t=0.14$.

The initial condition for Sod problem is

$$(\rho, u, p) = \begin{cases} (1, 0, 1) & 0 \leq x < 0.5 \\ (0.125, 0, 0.1) & 0.5 \leq x \leq 1, \end{cases} \quad (20)$$

and the final time is $t=0.2$.

The initial condition for the right half of Woodward and Colella problem is

$$(\rho, u, p) = \begin{cases} (1, 0, 0.01) & 0 \leq x < 0.5 \\ (1, 0, 100) & 0.5 \leq x \leq 1, \end{cases} \quad (21)$$

and the final time is $t=0.035$. Its solution contains a strong left shock, a contact discontinuity and a right rarefaction, as shown in Fig. 7(c).

We compare the results obtained by the third-order IDC (IDC3) method with what obtained by the widely used optimal SSP-RK3 in [29]. The problems are solved by using 400 spatial grid points. The reference solution, denoted by “Exact”, is computed by SSP-RK3 using 2000 spatial grid points. The comparison of the numerical results of density distribution is shown in Fig. 7. The areas with strong discontinuities are enclosed by magenta boxes and zoomed in for better observation. The results indicate that IDC3 can stably solve shock tube problems with different shock strength. It is also observed that IDC3 is slightly more stable than the SSP-RK3 method in all three cases. The results we obtained here may justify the use of IDC methods in flow problems with discontinuities. In fact, Sun and Xia [40] have used the IDC method to solve hyperbolic systems with discontinuities, and demonstrated that the method is asymptotic preserving.

Isentropic vortex advection

The first problem for studying parallel performance is the advection of an isentropic vortex, which is the same as the one used in Section 4.1. In this case, we use a spatial grid of the size 192×192 , and set the final time $t_e = 10$. The time grid consists of four intervals, each containing 60 sub-intervals, i.e., $N = 4$ and $M = 60$. Table 8 provides the wall time consumed by RIDC-ST of different orders and the speedup achieved. The speedup is over the IDC methods of the same order, which solve the problem step by step sequentially in time direction. Despite the diminishing efficiency as the order of accuracy increases, the fifth-order RIDC-ST method still gains a notable speedup of 3.45 when compared to the corresponding sequential approach. The sequential approach performs the prediction and corrections step by step. The fifth-order RIDC-ST only uses 801 seconds to converge, which is less than two times of the wall time consumed by the first-order RIDC-ST, but gains much more accurate results. The results are encouraging as it is known that parallelism in time direction is much harder than parallelism in space direction, especially for advection-dominated problems. The comparison of the results is shown in Fig. 8. Fig. 8(a) displays the density contours at the initial time, and Fig. 8(b) plots the density distributions along the center line at the initial time and end time. It is evident that RIDC-ST of fifth-order time accuracy yields significantly more accurate results than the first-order approach.

Similar to the comparison of RIDC³-ST and SSP-RK3 in Section 3.3, we compare the two methods for solving the isentropic vortex problem. A larger spatial grid of 384×384 is used to avoid the influence of spatial error. The final time is set to $t_e = 1$. SSP-RK3 uses 40 time steps, and RIDC³-ST adopts 3 time steps ($N = 3$) with $M = 40$. Since each of the SSP-RK3 step consists of three stages, the total number of stages of the 40 SSP-RK3 steps is 120, which is exactly the same as the number of the sub-intervals of RIDC³-ST using $N = 3$ and $M = 40$. Therefore, the comparison is considered to be fair.

The time consumed by SSP-RK3 to solve the problem is 1128s, while RIDC³-ST uses 1033s, which is slightly faster than SSP-RK3 by 8.4%. The L_2 -norm of the error of SSP-RK3 is 6.88×10^{-7} , while that of RIDC³-ST is 2.73×10^{-7} . Similar to the results in Section 3.3, RIDC³-ST obtains more accurate results than SSP-RK3 by using less wall time. We also compare the accuracy achieved by the two methods for a given amount of work. The two methods using various numbers of time steps are utilized to solve the problem. We specify the wall time consumed by SSP-RK3 of 40 time steps for solving the problem as one work unit, and then normalize other data. The results are drawn in Fig. 9. Again, it is observed that RIDC³-ST provides more accurate results than SSP-RK3 for a given amount of work unit in all cases for this problem.

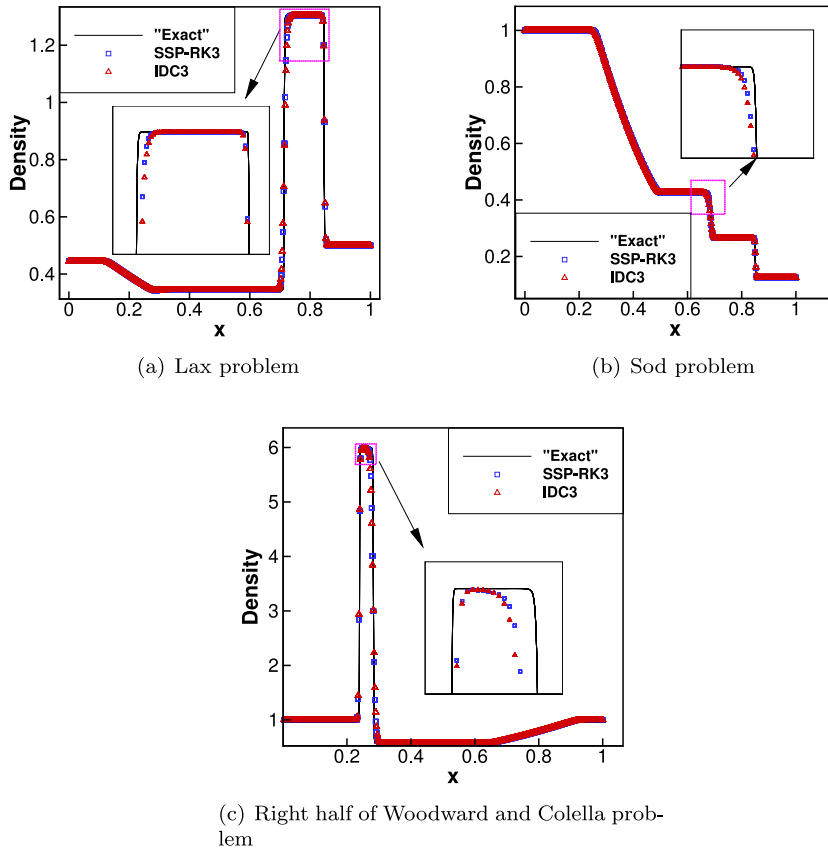


Fig. 7. Numerical results of density distribution of three shock tube problems computed by IDC3 and SSP-RK3. The area enclosed by a magenta box is enlarged for detailed observation. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 8

The isentropic vortex advection problem. Wall time consumed by RIDC-ST of different orders from first order to fifth order. “Seq. time” column in the table indicates the time consumed by the IDC methods of p th order.

p	Time (s)	Seq. time (s)	Speedup	Efficiency (%)
1	546	—	—	—
2	577	1096	1.90	95.0
3	627	1651	2.63	87.8
4	759	2207	2.91	72.7
5	801	2767	3.45	69.1

Two-dimensional Riemann problem

We consider the two-dimensional (2D) Riemann problem in [41]. The problem is generally used to research gas dynamics and to test numerical schemes. Initially, different conditions are imposed in four quadrants, so that discontinuities may occur at the interfaces. Specifically, we consider the configuration 16 in [41] and set the initial conditions as

$$(\rho, u, v, p) = \begin{cases} (0.5313, 0.1, 0.1, 0.4), & 0.5 \leq x \leq 1, 0.5 \leq y \leq 1, \\ (1.0222, -0.6179, 0.1, 1.0), & 0 \leq x \leq 0.5, 0.5 \leq y \leq 1, \\ (0.8, 0.1, 0.1, 1.0), & 0 \leq x \leq 0.5, 0 \leq y \leq 0.5, \\ (1.0, 0.1, 0.8276, 1.0), & 0.5 \leq x \leq 1, 0 \leq y \leq 0.5. \end{cases} \quad (22)$$

The resolution of the spatial grid is 400×400 , and the time grid consists of four intervals, each containing 60 sub-intervals, i.e., $N = 4$ and $M = 60$. At the end of the simulation time $t_e = 0.2$, three shocks and one rarefaction wave are generated, which are displayed in Fig. 10. The density contours computed by RIDC¹-ST and RIDC⁵-ST are displayed in Figs. 10(a) and 10(b), respectively. RIDC⁵-ST gives slightly sharper shock interface and more details, as is seen in the enlarged area. The speedup and parallel efficiency achieved by high order RIDC-ST methods are given in Table 9. Compared to the sequential computing, both the fourth-order and fifth-order time-parallel RIDC-ST achieve a speedup of approximately three, with the efficiencies being 67.4% and 56.8%, respectively. It is

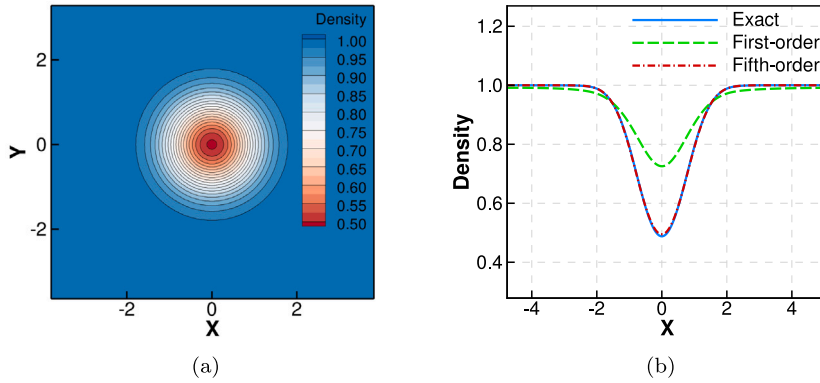


Fig. 8. Results for the isentropic vortex problem. (a) 21 equally spaced density contours between 0.5 and 1.0 at the initial time. (b) Density profiles at $y = 0$ of the first-order and fifth-order RIDC-ST and the exact solution.

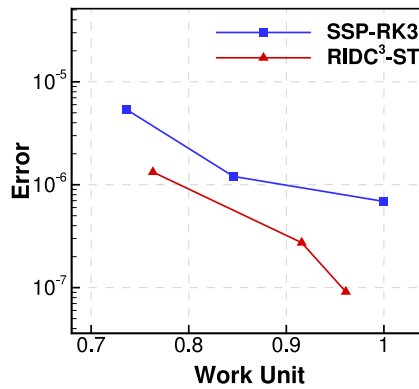


Fig. 9. Isentropic vortex advection problem. The accuracy achieved by the third-order RIDC-ST and SSP-RK3 by using certain amounts of work unit.

Table 9

The two-dimensional Riemann problem. Wall time consumed by RIDC-ST of different orders from first order to fifth order. “Seq. time” column in the table indicates the time consumed by the IDC methods of p th order.

p	Time (s)	Seq. time (s)	Speedup	Efficiency (%)
1	248	—	—	—
2	336	488	1.45	72.6
3	421	975	2.32	77.2
4	509	1372	2.70	67.4
5	617	1753	2.84	56.8

noticed that the high-order RIDC-ST methods consume much more time than the first-order one. This is because the high-order methods take more iterations (pseudo-time iterations that mentioned in the introduction of Section 4) to converge for this specific problem.

Flow over a circular cylinder

Flow over a circular cylinder is a classical problem that represents a widely existing natural phenomenon, i.e., the flow around a blunt body. We study this problem at the Reynolds number (Re) of 150. Periodic vortex shedding is observed after a certain period of time, which is known as Kármán vortex street, shown in Fig. 11(a). The spatial grid size is 176×84 . The temporal grid is set to $N = 2000$ and $M = 60$, and the end time is $t_e = 1200$.

In order to perform space-time parallel computing of this problem, spatial grid is split into eight blocks, as is shown in Fig. 11(a) by black thick lines. RIDC¹-ST needs 8 (8×1) processors in total for the computation of the problem, 8 in time direction and 1 in space direction. Similarly, RIDC⁵-ST requires 40 (8×5) processors in total, 8 in space direction and 5 in time direction. The oscillation of lift and drag coefficients computed by RIDC⁵-ST is drawn in Fig. 11(b). The calculated Strouhal number (which is defined as $S = fD/U$, where f is the vortex shedding frequency, D the diameter of the cylinder and U the incoming flow velocity) is 0.184, which agrees well with 0.184 in [42] and 0.18 in [43]. The averaged drag force coefficient is 1.363, which is close to 1.37 in [43].

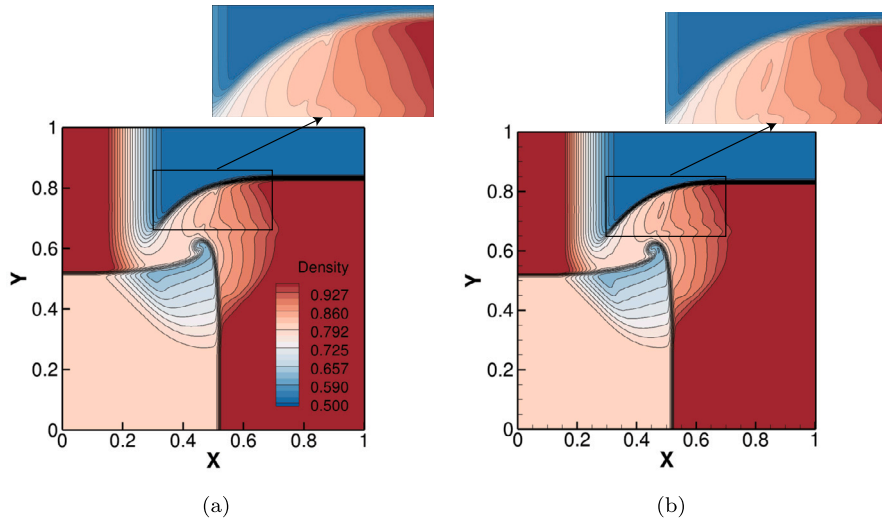


Fig. 10. Twenty-one equally spaced density contours from 0.5 to 0.95 of the 2D Riemann problem (configuration 16 in [41]) at $t = 0.2$. (a) RIDC¹-ST. (b) RIDC⁵-ST.

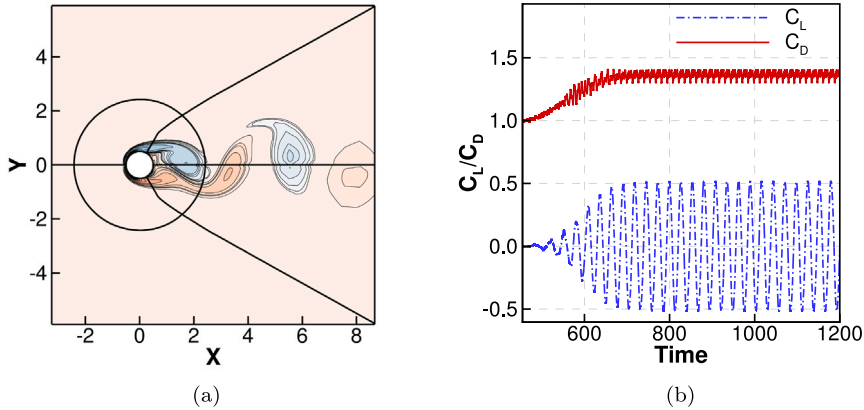


Fig. 11. Flow over a circular cylinder. (a) Vorticity contours at the time $t = 1200$. Mesh and mesh-splitting edges also shown in the figure. (b) Lift and drag force coefficients integrated on the surface of the cylinder during the computing time.

We consider the speedup achieved by high-order RIDC-ST for this problem. Space-time-parallel computing of this problem and the next sphere problem is performed on a cluster composed of several servers with two AMD EPYC 7742 64-Core CPUs. Wall time consumed by RIDC-ST methods is shown in Table 10. Since we are more concerned about time-parallel performance, we only compare the RIDC-ST methods using the same number of processors in space direction but different in time direction. The “Seq. time” columns in Table 10 means the time consumed by the IDC methods of p th order, where the problem is solved parallelly in space direction but sequentially in time direction, rather than sequentially in both directions. It is observed that the parallel efficiency is relatively high for this problem. Even RIDC⁵-ST shows a parallel efficiency of 69.5%. Both RIDC⁴-ST and RIDC⁵-ST provide more than three times of acceleration. In an alternative perspective, RIDC⁵-ST achieves the fifth-order accuracy, but only using slightly more wall time than that consumed by RIDC¹-ST.

We consider a scaling test for this problem to study the gains in using space-time parallel methods when parallel-only-in-space methods reach their speedup limits. The same grid used above is split into 256 blocks. We compute the flow over the cylinder for a short time for $t_e = 0.4$.

The IDC methods use 1 to 256 processors for spatial parallelism. The speedup they achieve are drawn in Fig. 12. It is observed that when more than 32 processors are used, the parallel efficiency falls rapidly. All the IDC methods reach their speedup limit when using 64 processors, providing around 50 times of speedup. After that, using more processors for spatial parallelism could not offer extra benefit. In terms of RIDC-ST, we use 64 processors for parallelism in space direction, and use 2 to 5 processors in time direction. For examples, RIDC⁵-ST utilizes 320 (64×5) processors in total for space-time parallel computing. The same total number of sub-intervals is used as in the IDC methods for a fair comparison. The four symbols in the upper right part of Fig. 12 represent the speedup achieved by RIDC-ST of order two to five. It is clear that RIDC-ST can break through the space-parallel bottleneck. Specific

Table 10

Flow over a circular cylinder. Wall time consumed by RIDC-ST of different orders from first order to fifth order. “Seq. time” column in the table indicates the time consumed by the IDC methods of p th order, parallel in space direction but sequential in time direction.

p	Time (h)	Seq. time (h)	Speedup	Efficiency (%)
1	5.05	–	–	–
2	5.32	10.04	1.89	94.4
3	5.64	16.67	2.96	98.5
4	6.58	20.27	3.08	77.0
5	7.23	25.10	3.47	69.5

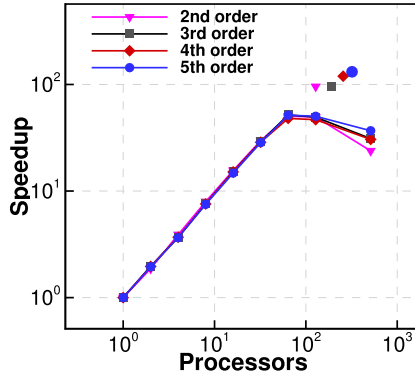


Fig. 12. Scaling test of the flow over a circular cylinder. The IDC methods utilize 1 to 256 processors for spatial parallelism. RIDC-ST methods of second order to fifth order can further accelerate the computation when the IDC methods reach their speedup limits, which are indicated by four individual markers in the upper right corner. The same color and shape represent the same order of accuracy for the two methods. RIDC-ST methods use 64 processors for parallelism in space direction, and 2 to 5 processors in time direction.

Table 11

Wall time consumed by RIDC-ST and the IDC methods using spatial parallelism, and the speedup they achieve. The IDC methods utilize 64 processors for space-parallel computing. The RIDC-ST methods use 64 processors for parallelism in space direction, and 2 to 5 processors in time direction.

Order	Time (s)				Speedup			
	2	3	4	5	2	3	4	5
IDC	18.4	43.6	77.2	102.4	50.4	51.8	48.0	51.5
RIDC-ST	9.7	23.5	31.0	40.1	95.6	96.1	119.6	131.5

numbers are collected in Table 11. It is seen that RIDC-ST could provide much more speedup than the IDC methods. For instance, RIDC⁵-ST achieves a speedup of 131.5, while the corresponding IDC method provides up to 51.5 times of acceleration. In general, the space–time parallel RIDC methods could further accelerate the computing for two to three times when the space-parallel IDC methods reach their speedup limits.

Flow over a sphere

We consider the problem of flow over a sphere at $Re = 300$ to test the performance of the RIDC-ST in engineer-level problems. The spatial grid is composed of 1.344 million hexahedron cells and is split into 21 blocks. Fig. 13 provides the global structured grid topology and the locally enlarged grid around the sphere. The front of the grid is cut open for the convenience of observation, and the grid-splitting edges are shown in black thick curves. It is seen in Fig. 13(a) that the wake area is refined to capture the vortex structure downstream. For this problem, we set the end of the computation time $t_e = 900$, and set the time grid parameters as $N = 400$ and $M = 50$. Since 21 processors are applied to perform parallel computing in spatial direction, RIDC⁵-ST requires 105 processors in total to perform space–time parallel computing.

Fig. 14 displays the results computed by RIDC⁵-ST. Fig. 14(a) shows the wake structure of the flow around a sphere using the Q-criterion method [44]. A sequence of shed hairpin vortices are clearly seen in the figure, similar to the results in [45]. Fig. 14(b) gives the front view of the wake structure, superimposed on the pressure contours around the sphere. The computed Strouhal number of 0.134 is in well agreement with that of Johnson and Patel [45] at 0.137 and that of Tomboulides [46] at 0.136. The calculated drag coefficient of 0.647 is close to the results of Johnson and Patel [45] at 0.656, and lies between the experimental results of Roos and Willmarth [47] at 0.629 and the numerical results of Tomboulides [46] at 0.671.

Finally, we focus on the parallel performance of RIDC-ST for this problem. It is seen in Table 12 that the fifth-order RIDC-ST achieves around three times of speedup compared to the one computed parallelly only in space direction. Other RIDC-ST methods with $p > 1$ also accelerate the computation, with the parallel efficiency larger than 50%. The results indicate that even

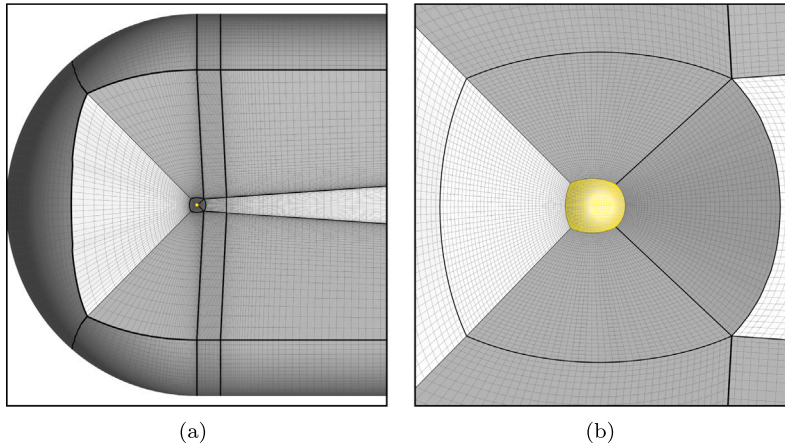


Fig. 13. The grid used to solve the flow over a sphere. The front of the grid is cut open for better observation. (a) Global structured grid topology, with grid-splitting edges displayed by black and thick curves. (b) The enlarged view of the grid around the sphere, with the sphere colored by yellow. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

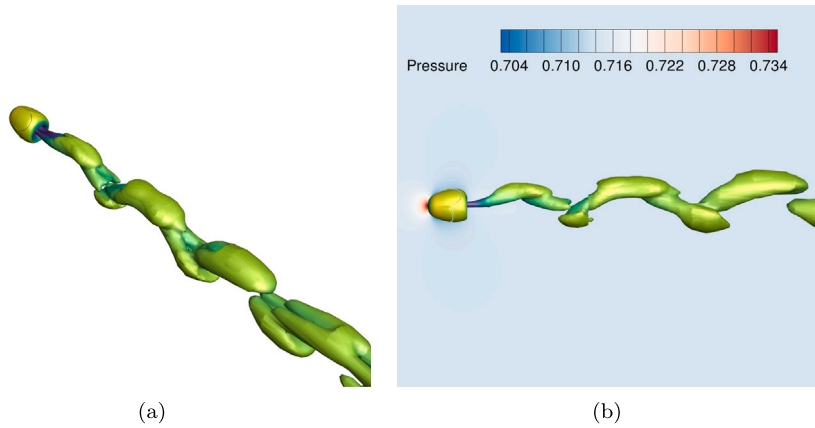


Fig. 14. Wake structure of the flow over a sphere displayed by Q-criterion. (a) A sequence of shed hairpin vortices. (b) The front view of the wake structure, superimposed on the pressure contours around the sphere.

Table 12

Flow over a sphere. Wall time consumed by RIDC-ST of different orders from first order to fifth order. “Seq. time” column in the table indicates the time consumed by the IDC methods of p th order, parallel in space direction but sequential in time direction.

p	Time (h)	Seq. time (h)	Speedup	Efficiency (%)
1	19.7	–	–	–
2	24.6	42.7	1.74	86.8
3	34.8	70.4	2.02	67.4
4	40.4	104.3	2.58	64.6
5	46.0	126.8	2.76	55.1

for problems with large numbers of degree of freedom, RIDC-ST should further accelerate the computation on the basis of spatial parallel acceleration.

It is worth noting that several problems in this section are advection-dominated, such as the isentropic vortex problem and two-dimensional Riemann problem. RIDC-ST did not encounter convergence difficulties for these problems. This property is desirable since many other PinT methods, such as PFASST and MGRIT, could degenerate when they are utilized to solve advection-dominated problems.

5. Conclusions

In this paper, we achieved high-order space–time parallel computing of the Navier–Stokes equations based on the RIDC algorithm. We integrated RIDC into a high-order compressible fluid solver using the MPI framework. The global MPI communicator is split into time communicators and space communicators, which manage the parallel communication in time and space directions, respectively.

One notable advantage of using RIDC is that unlike many other time-parallel methods such as PFASST and MGRIT, RIDC does not encounter convergence difficulties for convection-dominated problems. Another benefit of the method is that even when a few number of processors are available, they can be utilized for time-parallel acceleration. Both advantages are shown in the numerical examples in this paper. The accuracy and parallel performance of the method are demonstrated in model problems and unsteady flow problems. The method achieves the expected order of accuracy and times of speedup in time direction. Scaling tests of the flow over a circular cylinder illustrate that RIDC-ST is able to break through the spatial-parallel bottleneck. In addition, we compare the third-order RIDC-ST with the well-known strong stability preserving Runge–Kutta method. It is found that the proposed method is more accurate and slightly faster than the latter. The space–time parallel computing of the flow over a sphere demonstrates that space–time parallel RIDC has the potential to accelerate the computation even for engineering level problems.

CRediT authorship contribution statement

Meiyuan Zhen: Writing – review & editing, Writing – original draft, Visualization, Investigation, Formal analysis, Data curation. **Xuan Liu:** Validation. **Xuejun Ding:** Software, Resources. **Jinsheng Cai:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The authors acknowledge the support from the National Natural Science Foundation of China (Grant No. 91952203).

References

- [1] Jürg Nievergelt, Parallel methods for integrating ordinary differential equations, *Commun. ACM* 7 (12) (1964) 731–733.
- [2] Martin J. Gander, 50 years of time parallel time integration, in: Thomas Carraro, Michael Geiger, Stefan Körkel, Rolf Rannacher (Eds.), in: *Multiple Shooting and Time Domain Decomposition Methods*, vol. 9, Springer International Publishing, Cham, 2015, pp. 69–113.
- [3] Benjamin W. Ong, Jacob B. Schroder, Applications of time parallelization, *Comput. Vis. Sci.* 23 (1–4) (2020) 11.
- [4] Kevin Burrage, Parallel methods for initial value problems, *Appl. Numer. Math.* 11 (1–3) (1993) 5–25.
- [5] Thi-Thao-Phuong Hoang, Jérôme Jaffré, Caroline Japhet, Michel Kern, Jean E. Roberts, Space-time domain decomposition methods for diffusion problems in mixed formulations, *SIAM J. Numer. Anal.* 51 (6) (2013) 3532–3559.
- [6] Stefan Vandewalle, Dirk Roose, The parallel waveform relaxation multigrid method, in: *Parallel Processing for Scientific Computing*, SIAM, 1989, pp. 152–156.
- [7] Jacques-Louis Lions, Yvon Maday, Gabriel Turinici, Résolution d'EDP par un schéma en temps pararéel, *C. R. l'Acad. Sci.-Ser. I-Math.* 332 (7) (2001) 661–668.
- [8] Matthew Emmett, Michael Minion, Toward an efficient parallel in time method for partial differential equations, *Commun. Appl. Math. Comput. Sci.* 7 (1) (2012) 105–132.
- [9] R.D. Falgout, S. Friedhoff, Tz. V. Kolev, S.P. MacLachlan, J.B. Schroder, Parallel time integration with multigrid, *SIAM J. Sci. Comput.* 36 (6) (2014) C635–C661.
- [10] Thibaut Lunet, Julien Bodart, Serge Gratton, Xavier Vasseur, Time-parallel simulation of the decay of homogeneous turbulence using Parareal with spatial coarsening, *Comput. Vis. Sci.* 19 (1–2) (2018) 31–44.
- [11] Daniel Ruprecht, Rolf Krause, Explicit parallel-in-time integration of a linear acoustic-advection system, *Comput. & Fluids* 59 (2012) 72–83.
- [12] Andreas Henthaler, Robert D. Falgout, Jacob B. Schroder, Adelaide de Vecchi, David Nordsletten, Oliver Röhrle, Time-periodic steady-state solution of fluid-structure interaction and cardiac flow problems through multigrid-reduction-in-time, *Comput. Methods Appl. Mech. Engrg.* 389 (2022) 114368.
- [13] Martin J. Gander, Felix Kwok, Julien Salomon, PARAOPT: A parareal algorithm for optimality systems, *SIAM J. Sci. Comput.* 42 (5) (2020) A2773–A2802.
- [14] Weifan Liu, Minghao W. Rostami, Parallel-in-time simulation of biofluids, *J. Comput. Phys.* (2022) 111366.
- [15] Joshua Christopher, Robert D. Falgout, Jacob B. Schroder, Stephen M. Guzik, Xinfeng Gao, A space-time parallel algorithm with adaptive mesh refinement for computational fluid dynamics, *Computing and Visualization in Science* 23 (1–4) (2020) 13.
- [16] Xuhui Meng, Zhen Li, Dongkun Zhang, George Em Karniadakis, PPINN: Parareal physics-informed neural network for time-dependent PDEs, *Comput. Methods Appl. Mech. Engrg.* 370 (2020) 113250.
- [17] Alexander J. Howse, Hans De Sterck, Robert D. Falgout, Scott MacLachlan, Jacob Schroder, Parallel-in-time multigrid with adaptive spatial coarsening for the linear advection and inviscid burgers equations, *SIAM J. Sci. Comput.* 41 (1) (2019) A538–A565.
- [18] Hans De Sterck, Robert D. Falgout, Stephanie Friedhoff, Oliver A. Krzysik, Scott P. MacLachlan, Optimizing multigrid reduction-in-time and parareal coarse-grid operators for linear advection, *Numer. Linear Algebra Appl.* 28 (4) (2021) e2367.
- [19] Andrew J. Christlieb, Colin B. Macdonald, Benjamin W. Ong, Parallel high-order integrators, *SIAM J. Sci. Comput.* 32 (2) (2010) 818–835.
- [20] Alok Dutt, Leslie Greengard, Vladimir Rokhlin, Spectral deferred correction methods for ordinary differential equations, *BIT Numer. Math.* 40 (2000) 241–266.
- [21] Andrew Christlieb, Benjamin Ong, Jing-Mei Qiu, Comments on high-order integrators embedded within integral deferred correction methods, *Commun. Appl. Math. Comput. Sci.* 4 (1) (2009) 27–56.
- [22] Yinhua Xia, Yan Xu, C. Shu, Efficient time discretization for local discontinuous Galerkin methods, *Discrete Contin. Dyn. Syst. Ser. B* 8 (3) (2007) 677.
- [23] Andrew Christlieb, Benjamin Ong, Implicit parallel time integrators, *J. Sci. Comput.* 49 (2011) 167–179.

- [24] Andrew Christlieb, Colin Macdonald, Benjamin Ong, Raymond Spiteri, Revisionist integral deferred correction with adaptive step-size control, *Commun. Appl. Math. Comput. Sci.* 10 (1) (2015) 1–25.
- [25] Wenjia Liu, Yanren Hou, Dandan Xue, Numerical analysis of a 4th-order time parallel algorithm for the time-dependent Navier-Stokes equations, *Appl. Numer. Math.* 150 (2020) 361–383.
- [26] Andrew J. Christlieb, Ronald D. Haynes, Benjamin W. Ong, A parallel space-time algorithm, *SIAM J. Sci. Comput.* 34 (5) (2012) C233–C248.
- [27] Benjamin W. Ong, Ronald D. Haynes, Kyle Ladd, Algorithm 965: RIDC methods: A family of parallel time integrators, *ACM Trans. Math. Softw.* 43 (1) (2016) 1–13.
- [28] Guang-Shan Jiang, Chi-Wang Shu, Efficient implementation of weighted ENO schemes, *J. Comput. Phys.* 126 (1) (1996) 202–228.
- [29] Sigal Gottlieb, David I. Ketcheson, Chi-Wang Shu, High order strong stability preserving time discretizations, *J. Sci. Comput.* 38 (3) (2009) 251–289.
- [30] Fei Liao, Zhengyin Ye, Lingxia Zhang, Extending geometric conservation law to cell-centered finite difference methods on stationary grids, *J. Comput. Phys.* 284 (2015) 419–433.
- [31] Xiaogang Deng, Hanxin Zhang, Developing high-order weighted compact nonlinear schemes, *J. Comput. Phys.* 165 (1) (2000) 22–44.
- [32] Yasuhiro Wada, Meng-Sing Liou, A flux splitting scheme with high-resolution and robustness for discontinuities, in: 32nd Aerospace Sciences Meeting and Exhibit, 1994, p. 83.
- [33] Antony Jameson, Time dependent calculations using multigrid, with applications to unsteady flows past airfoils and wings, in: 10th Computational Fluid Dynamics Conference, 1991, p. 1596.
- [34] Thomas H. Pulliam, D.S. Chaussee, A diagonal form of an implicit approximate-factorization algorithm, *J. Comput. Phys.* 39 (2) (1981) 347–363.
- [35] H.C. Yee, N.D. Sandham, M.J. Djomehri, Low-dissipative high-order shock-capturing methods using characteristic-based filters, *J. Comput. Phys.* 150 (1) (1999) 199–238.
- [36] Chi-Wang Shu, Stanley Osher, Efficient implementation of essentially non-oscillatory shock-capturing schemes, *J. Comput. Phys.* 77 (2) (1988) 439–471.
- [37] Peter D. Lax, Weak solutions of nonlinear hyperbolic equations and their numerical computation, *Commun. Pure Appl. Math.* 7 (1) (1954) 159–193.
- [38] Gary A. Sod, A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws, *J. Comput. Phys.* 27 (1) (1978) 1–31.
- [39] Paul Woodward, Phillip Colella, The numerical simulation of two-dimensional fluid flow with strong shocks, *J. Comput. Phys.* 54 (1) (1984) 115–173.
- [40] Chong Sun, Yinhua Xia, Asymptotic preserving spectral deferred correction methods for hyperbolic systems with relaxation, *Commun. Comput. Phys.* 26 (2) (2019) 531–557.
- [41] Peter D. Lax, Xu-Dong Liu, Solution of two-dimensional Riemann problems of gas dynamics by positive schemes, *SIAM J. Sci. Comput.* 19 (2) (1998) 319–340.
- [42] C.H.K. Williamson, Vortex dynamics in the cylinder wake, *Annu. Rev. Fluid Mech.* 28 (1996) 477–539.
- [43] A.L.F. Lima E. Silva, A. Silveira-Neto, J.J.R. Damasceno, Numerical simulation of two-dimensional flows over a circular cylinder using the immersed boundary method, *J. Comput. Phys.* 189 (2) (2003) 351–370.
- [44] Julian C.R. Hunt, Alan A. Wray, Parviz Moin, Eddies, streams, and convergence zones in turbulent flows, in: *Studying Turbulence Using Numerical Simulation Databases, 2. Proceedings of the 1988 Summer Program*, 1988.
- [45] T.A. Johnson, V.C. Patel, Flow past a sphere up to a Reynolds number of 300, *J. Fluid Mech.* 378 (1999) 19–70.
- [46] Ananias Georgios Tomboulides, Direct and Large Eddy Simulation of Wake Flows: Flow Past a Sphere, Princeton University, 1993.
- [47] Frederick W. Roos, William W. Willmarth, Some experimental results on sphere and disk drag, *AIAA J.* 9 (2) (1971) 285–291.